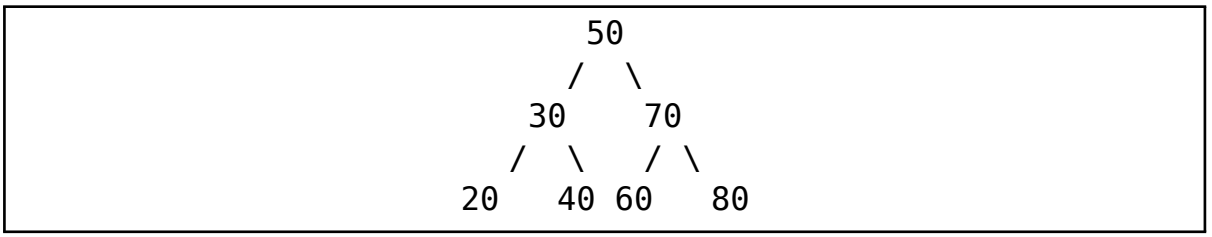
KEMENTERIAN RISET, TEKNOLOGI, DAN PENDIDIKAN TINGGI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" YOGYAKARTA
JL. SWK 104 (Lingkar Utara) Condongcatur, Yogyakarta 55283 Telp.(0274)486188, 486733, Fax. 486400
Jl. Babarsari 2, Tambakbayan, Yogyakarta 55281 Telp.(0274)486911
Email : info@upnyk.ac.id. Homepage : http://www.upnyk.ac.id

KUIS MATA KULIAH PRAKTIKUM SEMESTER GASAL TA. 2026/2027
PRAKTIKUM IMPLEMENTASI STRUKTUR DATA
PERTEMUAN 08 - Binary Search Tree
KERJAKAN SESUAI NAMA FUNCTION PADA STUDENT.CPP

STUDY CASE

Pada Pertemuan 7, *`left`* dan *`right`* cuma menyatakan POSISI, tidak ada aturan besar-kecil sama sekali. Pertemuan ini menambahkan satu aturan: semua nilai di cabang KIRI sebuah node lebih KECIL daripada nilai node itu, dan semua nilai di cabang KANAN sebuah node lebih BESAR daripada nilai node itu. Aturan itu disebut INVARIANT: sesuatu yang harus SELALU benar, sebelum maupun sesudah setiap operasi.

Tree contoh yang dipakai di hampir seluruh soal pada pertemuan ini: tree ini terbentuk bila nilai 50, 30, 70, 20, 40, 60, 80 dimasukkan berturut-turut ke dalam tree kosong.



`inorder(root)` mencetak: *`20 30 40 50 60 70 80`*. Hasil inorder pada BST yang sah SELALU terurut naik. Sifat itu dipakai terus sebagai pemeriksaan cepat: kalau sesudah insert atau hapus hasil inorder Anda tidak lagi naik, pasti ada invariant yang rusak.

PENTING: aturannya bukan "child kiri lebih kecil dari parent-nya", melainkan "SELURUH nilai di cabang kiri lebih kecil", berlaku sampai sedalam-dalamnya. Karena aturannya selalu benar, satu kali perbandingan langsung membuang separuh kemungkinan. Kelima fungsi yang Anda kerjakan memakai gagasan yang sama: bandingkan, lalu pilih SATU cabang. Tidak ada satu pun yang perlu memeriksa kedua cabang.

Pada pertemuan ini sebuah nilai hanya boleh muncul SEKALI di dalam tree. Nilai yang sudah ada ditolak oleh *`insert`*. Dengan begitu aturan "kiri lebih kecil, kanan lebih besar" tidak pernah ambigu.

ISTILAH YANG DIPAKAI DI SELURUH SOAL		
`node`	Satu kotak penyimpanan: sebuah angka (data), alamat cabang kiri (left), dan alamat cabang kanan (right)	**Struct**
`root`	Alamat node paling atas. Bernilai nullptr berarti tree sedang KOSONG	**Pointer**
`subtree`	Bagian tree yang berakar pada suatu node; invariant-nya juga berlaku di sana	**Konsep**
`successor`	Nilai terkecil yang masih lebih besar daripada sebuah node (dipakai pada Soal 2b)	**Konsep**

Berikut daftar seluruh soal pada pertemuan ini. Lima fungsi dinilai, dan dua fungsi lain (`*inorder*` dan `*clear*`) sudah disediakan lengkap di bagian bawah file, tidak dinilai.

1. Soal 1 — Invariant & Pencarian: *insert* dan *cari*. **(35 poin)**
2. Soal 2 — Minimum & Penghapusan: *minimum* dan *hapus*. **(40 poin)**
3. Soal 3 — Soal cerita: *termahalTerbeli*. **(25 poin)**

1a. insert **20 Poin**

<table>
<tr><td>Quest</td><td colspan="2">Sebuah nilai baru masuk ke dalam tree. Tugasnya menempatkan nilai itu di posisi yang BENAR, sehingga aturan kiri-lebih-kecil-kanan-lebih-besar tetap berlaku sesudahnya. Cara menemukan posisinya sama persis dengan cara mencari: bandingkan nilai baru dengan node yang sedang dipegang, lalu pilih satu cabang. Bedanya, penelusuran ini tidak berhenti karena ketemu, melainkan berhenti karena sampai di tempat KOSONG. Tempat kosong itulah milik node baru.

Nilai yang SUDAH ADA ditolak: tree tidak boleh berubah sama sekali, dan tidak boleh ada node baru yang dibuat. Bentuk cabang yang sudah ada tidak boleh diubah, dipindah, atau disusun ulang.</td></tr>
<tr><td rowspan="3">Parameter/ Variabel</td><td><code>root</code></td><td>Alamat node paling atas tree milik pemanggil. Bertanda `&`, karena pada tree kosong `root` milik pemanggil ikut berubah menjadi node baru itu. Boleh bernilai `nullptr`</td></tr>
<tr><td><code>nilai</code></td><td>Angka yang mau dimasukkan</td></tr>
<tr><td><code>return</code></td><td>`<i>true</i>` bila nilai benar-benar masuk, `<i>false</i>` bila ditolak karena nilainya sudah ada</td></tr>
<tr><td>Gambaran</td><td colspan="2"><pre>Tree kosong:
Operasi : insert(root, 42)
Sesudah : true, dan root sekarang menunjuk node 42

Nilai duplikat ditolak, tree tidak berubah:
Operasi : insert(root, 40)
Sesudah : false -- 40 sudah ada
Operasi : insert(root, 55)
Sesudah : true   -- 55 belum ada, masuk sebagai cabang kiri milik 60</pre></td></tr>
</table>

1b. cari **15 Poin**

Quest	Pemanggil ingin tahu apakah sebuah nilai ada di dalam tree, dan kalau ada, ia ingin memegang NODE-nya, bukan sekadar jawaban "ada" atau "tidak ada". Karena invariant selalu berlaku, pencarian tidak perlu memeriksa semua node: cukup bandingkan nilai yang dicari dengan node yang sedang dipegang, lalu pilih satu cabang saja. Cabang yang lain dijamin tidak mungkin berisi nilai itu. Perhatikan arah perbandingannya baik-baik: nilai yang LEBIH KECIL ada di cabang KIRI. Mencari tidak boleh mengubah tree sedikit pun.	
Parameter/ Variabel	`root`	Alamat node paling atas tree yang mau dicari. Boleh bernilai `nullptr`, artinya tree sedang kosong
	`nilai`	Angka yang sedang dicari
	`return`	Alamat node yang data-nya sama dengan *`nilai`*, atau *`nullptr`* bila nilainya tidak ada

Gambaran	`Operasi : cari(root, 60)`
	`Sesudah : node 60  -- 60>50 kanan, 60<70 kiri, 60==60 ketemu`
	`Operasi : cari(root, 25)`
	`Sesudah : nullptr  -- 25 tidak ada di tree`
	`Operasi : cari(nullptr, 10)`
	`Sesudah : nullptr  -- tree-nya kosong`	

2a. minimum **10 Poin**

Quest	Pemanggil ingin tahu node mana yang menyimpan nilai TERKECIL. Pada BST, nilai terkecil selalu berada di tempat yang sama: selama masih ada cabang kiri, pasti masih ada nilai yang lebih kecil. Fungsi ini juga dipakai lagi pada Soal 2b, untuk mencari pengganti node yang dihapus. Fungsi ini boleh dipanggil pada subtree mana pun, bukan cuma pada *`root`*. Node yang tidak punya cabang kiri berarti dialah yang terkecil.	
Parameter/ Variabel	`root`	Alamat node paling atas tree ATAU subtree yang mau diperiksa. Boleh bernilai `nullptr`, artinya kosong
	`return`	Alamat node yang nilainya paling kecil, atau *`nullptr`* bila kosong

Gambaran	`Operasi : minimum(root)`
	`Sesudah : node 20  -- node paling kiri`
	`Operasi : minimum(root->right)`
	`Sesudah : node 60  -- paling kiri DI DALAM subtree kanan (subtree 70)`
	`Operasi : minimum(nullptr)`
	`Sesudah : nullptr  -- kosong`	

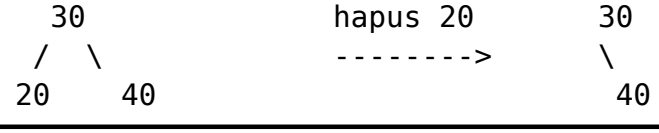
2b. hapus **30 Poin**

Quest	Sebuah nilai dikeluarkan dari tree: node yang menyimpannya dilepas lalu dibuang dari memori, dan sesudah itu tree harus TETAP memenuhi invariant. Menemukan node targetnya sama persis dengan *`cari`*. Yang menggantikan tempatnya bergantung pada BERAPA CABANG node target: 0 child (node langsung dilepas), 1 child (cabang satu-satunya naik menggantikan tempatnya), atau 2 child (penggantinya adalah INORDER SUCCESSOR, yaitu nilai terkecil pada cabang kanan node target, dicari dengan *`minimum`*). Pada keadaan 2 child: nilai successor dipindahkan ke node target, lalu node successor yang LAMA dihapus dari cabang kanan (node successor dijamin tidak pernah punya cabang kiri, jadi penghapusannya kembali ke keadaan 0 atau 1 child). TEPAT SATU node dibebaskan dengan *`delete`* pada setiap penghapusan yang berhasil.	
Parameter/ Variabel	`root`	Alamat node paling atas tree milik pemanggil. Bertanda `&`, sehingga tree yang berubah bentuk atau menjadi kosong ikut terasa oleh pemanggil. Boleh bernilai `nullptr`
	`nilai`	Angka yang mau dikeluarkan
	`return`	*`true`* bila memang ada node yang dihapus, *`false`* bila nilainya tidak ada

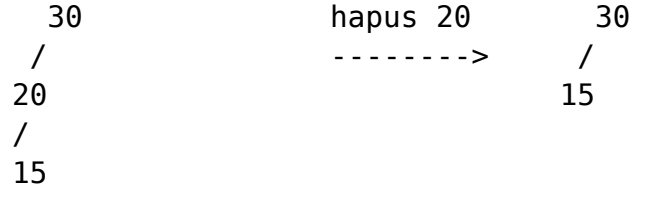
Gambaran	`Operasi : hapus(root, 20)`
	`Sesudah : true  -- 0 child. inorder: 30 40 50 60 70 80`
	`Operasi : hapus(root, 30)`
	`Sesudah : true  -- 2 child, successor-nya 40`
	`Operasi : hapus(root, 50)`
	`Sesudah : true  -- 2 child pada root, successor-nya 60`
	`Operasi : hapus(root, 25)`
	`Sesudah : false  -- 25 tidak ada, tree tidak berubah`	

Tiga Keadaan Penghapusan pada Soal 2b

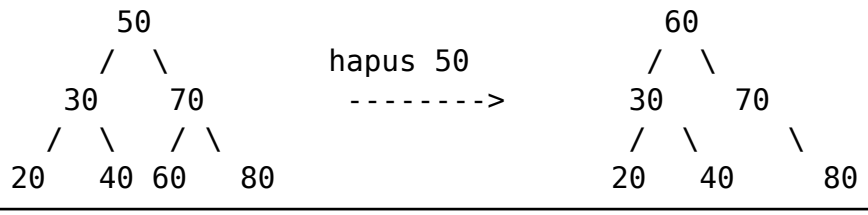
KEADAAN 1 — 0 child



KEADAAN 2 — 1 child: cabang satu-satunya NAIK



KEADAAN 3 – 2 child: diganti INORDER SUCCESSOR



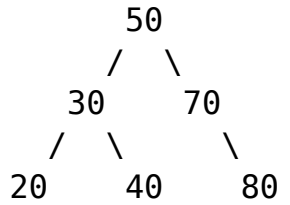
Successor-nya adalah nilai terkecil pada cabang kanan node target (di sini: 60), dipindahkan menggantikan 50, lalu node 60 yang lama dihapus dari cabang kanan. *`inorder`* sesudahnya: *`20 30 40 60 70 80`* (tetap terurut naik).

3. termahalTerbeli — SOAL CERITA **25 Poin**

Quest	Sebuah toko menyimpan daftar harga barangnya di dalam BST. Anda datang membawa uang sebanyak *`uang`*, dan ingin membeli barang yang PALING MAHAL yang harganya masih sanggup Anda bayar. Barang yang harganya PERSIS SAMA dengan uang Anda masih terhitung terbeli. Kalau semua barang kemahalan, Anda pulang dengan tangan kosong. Petunjuk cara berpikirnya: saat Anda sedang berdiri di sebuah harga, harga itu masih terjangkau atau sudah kemahalan. Masing-masing memberi tahu Anda di mana jawabannya mungkin berada, dan sama seperti soal-soal sebelumnya, Anda tidak perlu memeriksa kedua cabang. Menemukan satu harga yang terjangkau belum berarti pekerjaan selesai: pastikan tidak ada yang lebih mahal dan masih terbeli.	
Parameter/ Variabel	`root`	Alamat node paling atas daftar harga. Boleh bernilai `nullptr`, artinya tokonya sedang tidak punya barang sama sekali
	`uang`	Banyaknya uang yang Anda bawa
	`return`	Alamat node dengan harga termahal yang masih terbeli, atau *`nullptr`* bila tidak ada satu pun yang terbeli

Gambaran	`Operasi : termahalTerbeli(root, 65)`
	`Sesudah : 50 -- terjangkau: 20,30,40,50. Termahal: 50`
	`Operasi : termahalTerbeli(root, 45)`
	`Sesudah : 40 -- terjangkau: 20,30,40. Termahal: 40`
	`Operasi : termahalTerbeli(root, 99)`
	`Sesudah : 80 -- semua terjangkau, jawabannya yang termahal`
	`Operasi : termahalTerbeli(root, 15)`
	`Sesudah : nullptr -- bahkan barang termurah (20) sudah kemahalan`	

Daftar harga sebuah toko (dalam ribuan rupiah) yang dipakai pada contoh-contoh di atas:



CATATAN & ATURAN

- Signature fungsi dan bentuk *`struct Node`* adalah kontrak dan tidak boleh diubah. Isi fungsi, nama variabel, dan struktur kode di dalamnya sepenuhnya bebas, rekursif maupun memakai loop, keduanya diterima.
- Anda boleh menambah fungsi bantu sendiri.
- Anda BOLEH menulis *`main()`* sendiri di file ini, di bagian paling bawah yang sudah disediakan. Pakai *`cin`* dan *`cout`* sebebasnya di sana.
- Setiap soal dinilai sendiri-sendiri. Checker membangun tree ujinya tanpa memakai *`insert`* Anda, jadi Soal 1 yang belum benar tidak ikut menjatuhkan nilai Soal 2 dan Soal 3.
- Nilai duplikat selalu ditolak oleh *`insert`*; sebuah nilai hanya boleh muncul SEKALI di dalam tree.
- Dua fungsi, *`inorder`* dan *`clear`*, sudah disediakan lengkap dan tidak dinilai. *`inorder`* sangat berguna sebagai pemeriksa cepat: pada BST yang sah hasilnya SELALU terurut naik.
- *`cin`* hanya boleh dipakai DI DALAM *`main()`*. Jangan menaruh *`cin`* di dalam kelima fungsi yang dinilai, karena checker memanggil fungsi-fungsi itu tanpa memberi masukan apa pun.